ForenSight

Full Build Plan: the best approach to build the complete project

Explainable multi-signal deepfake forensics for images, videos and audio · 13 October 2026

1. The approach in one paragraph

Build ForenSight as a **pipeline of small, independent detectors** whose results are fused into one verdict and explained in plain language. Do **not** train big models from scratch: start every detector from a **proven pretrained model or a classic forensic technique**, get one complete upload-to-result path working end to end, then make each detector stronger one at a time. Build in the order **image** → **audio** → **video**, because each step reuses the previous one (video = image detectors on frames + audio detectors on the soundtrack + checks over time). Everything runs in Docker, and the demo is planned from day one.

2. Guiding principles

- **Vertical slice first.** One file type working end to end (upload → detect → verdict → result page) beats four half-built modules. Judges see a working product.
- **Pretrained first, fine-tune second, train from scratch never.** Hackathon time and a laptop CPU cannot train deepfake detectors from zero. Free GPUs (Google Colab, Kaggle) are enough for fine-tuning.
- **Every detector must give a reason, not just a number.** Each one returns a score, a reliability, plain-language reasons, and where possible a heatmap or time segments. This is ForenSight's main selling point.
- **Generalisation over benchmark accuracy.** Detectors often score 99% on their training dataset and fail on new generators. Always test on a dataset the model has *not* seen.
- **Honest uncertainty.** When detectors disagree or the file is heavily compressed, say **Inconclusive**. A wrong confident answer is worse than no answer.
- **CPU-friendly inference.** The demo must run on a normal laptop inside Docker, so pick models that run in seconds on CPU and load them once per worker.

3. System architecture

The skeleton for this already exists in `~/Desktop/ForenSight`. A request flows like this:

```
Browser → FastAPI (save file, create DB row) → Redis queue → Celery worker: preprocess →
run detectors → fuse → explain → PDF → MySQL → Browser polls and shows the result
```

Service	Role	Why this choice
frontend	React + Vite + Tailwind: upload, progress, result, history pages	Fast to build, easy charts (Recharts) and waveforms (wavesurfer.js)
backend	FastAPI: upload, status, results, report download, delete	Python like the ML code; automatic API docs
worker	Celery: runs the heavy pipeline, same image as the backend	Video takes seconds to minutes; must not block HTTP
redis	Job queue and result store	Standard pairing with Celery
mysql	Analysis history: verdict, confidence, evidence JSON, file paths	Required by the proposal
ml (optional)	Jupyter for experiments and evaluation	Same libraries as the worker; real training on Colab or Kaggle

Detector contract (the key design decision)

Every detector implements the same interface and returns a **DetectorResult**: *fake_score* (0 = real, 1 = fake), *reliability* (how much to trust it for this file), *fake_kind* (AI-generated or manipulated), *reasons* (sentences), and optionally a *heatmap* or *time segments*. Adding a new detector later means writing one class. Nothing else changes.

4. Best approach for each detector

4.1 Preprocessing (shared by all)

- Detect the file type from its **bytes** (magic numbers), not the extension. Reject anything else. Enforce a size limit.
- Images: load with Pillow and keep the original bytes for the metadata and ELA checks.
- Video: use ffmpeg to sample frames at 1-2 fps (up to about 64 frames) and extract the audio track to 16 kHz mono WAV.
- Faces: detect with **MTCNN** (facenet-pytorch) or RetinaFace, crop with a margin of about 30%, and track faces across frames.
- Audio: resample to 16 kHz mono and cut into about 4 s windows with overlap. Each window gets its own score, which builds the timeline.

4.2 Image

Check	Best approach	Output / reason example	Effort
Metadata + C2PA	Read EXIF with Pillow/exiftool (camera model, software tag, dates); look for AI-tool signatures; verify Content Credentials with the **c2pa-python** library.	"File was saved by Adobe Photoshop" · "Content Credentials say: generated by an AI tool"	Low
Error Level Analysis (ELA)	Re-save the JPEG at a known quality and take the difference. Edited regions recompress differently. Only meaningful for JPEG.	Heatmap of the edited region · "Area near the face edge compresses differently"	Low
Frequency (FFT) artifacts	Power spectrum of the noise residual. Generator upsampling leaves periodic peaks. Feed spectrum features to a small logistic-regression model trained on CIFAKE/GenImage samples.	"Repeating frequency pattern typical of AI generators"	Low-Med
Learned AI-image classifier	**CLIP ViT-L/14 features + linear probe** (the UnivFD approach, Ojha et al., CVPR 2023). Generalises to unseen generators much better than a CNN trained on one generator.	"Image matches the visual fingerprint of AI-generated images"	Medium
Face-swap classifier	**EfficientNet** (B0-B4 for CPU) fine-tuned on face crops from FaceForensics++ and Celeb-DF. **Grad-CAM** (pytorch-grad-cam) shows where the model looked.	Heatmap on jawline / blending boundary · "Blending artifacts around the face"	Medium

4.3 Audio

Check	Best approach	Output / reason example	Effort
Anti-spoofing model	Start from **AASIST** (pretrained on ASVspoof 2019 LA; official code in clovaai/aasist). For better generalisation, use a **wav2vec 2.0 / XLS-R** self-supervised front end with a small classifier fine-tuned on ASVspoof + WaveFake.	Per-window scores → timeline · "Voice in 0:04-0:11 matches cloned-speech patterns"	Medium
Splice / cut detection	Find sudden jumps in energy, background noise floor and spectral shape between neighbouring windows.	"Abrupt change in background noise at 0:17 suggests audio was joined"	Low
Prosody / breathing heuristics	Pitch variance, pause lengths and breath sounds. A weak signal, so give it low reliability.	"Speech has almost no breathing pauses"	Low

4.4 Video

Check	Best approach	Output / reason example	Effort
Per-frame face classifier	Reuse the image face-swap model on every sampled face crop, then average per second. (The DFDC-winning solution used an EfficientNet ensemble on face crops.)	Per-second timeline · heatmap on worst frame	Low (reuse)
Temporal consistency	Variance of frame scores, landmark jitter and face-box flicker between consecutive frames.	"Face edges flicker between 0:05 and 0:09"	Medium
Blink / head pose / lighting	Eye Aspect Ratio from MediaPipe Face Mesh landmarks for blink rate; head-pose angles; brightness of face vs. background. Weak signals, so low reliability.	"Person blinks far less than normal"	Medium
Lip-sync	**SyncNet**-style audio-visual sync score (pretrained) between the mouth region and the speech. Treat as a stretch goal.	"Lip movement does not match the voice 0:12-0:18"	High
Audio track	Run the full audio pipeline on the extracted soundtrack.	Merged into the same timeline	Low (reuse)

4.5 Fusion (combining the checks)

- **Now:** a reliability-weighted average with an Inconclusive band (already written in `fusion/combiner.py`).
- **Later:** once detectors are evaluated, train a small **logistic regression** on detector outputs from a validation set and **calibrate** it (Platt scaling) so "80% confidence" really means about 80%.
- Pick the Inconclusive thresholds on the validation set, not by guesswork.
- Decide AI-Generated vs Manipulated from *which* detectors fired: whole-image generators point to AI-Generated; ELA, face-swap and splice point to Manipulated.

4.6 Explanation

- Build reasons from **templates** filled with real detector values (times, regions, numbers). Templates are deterministic and never invent evidence.
- Show the top 3 reasons by contribution, the heatmap with an opacity slider, and a colour-coded timeline that seeks the video or audio when clicked.
- Optional later: an LLM can rephrase the template output into a friendlier summary, but it must only rephrase the given facts.

5. Datasets and training strategy

Dataset	Use for	Access
FaceForensics++	Face-swap / reenactment training (image + video)	Request form
Celeb-DF (v2)	Hard face-swap test set; cross-dataset evaluation	Request form
DFDC	Large, varied video deepfakes	Kaggle / Meta licence
CIFAKE	Real vs. AI images (quick experiments)	Kaggle, open
GenImage	AI images from many generators (generalisation)	Open download
ASVspoof 2019 LA / 2021	Voice-spoofing training and evaluation	Registration
WaveFake	Neural vocoder fake speech	Open (Zenodo)
In-the-Wild	Real-world audio deepfakes: generalisation test	Open

Apply for the gated datasets this week. FaceForensics++, Celeb-DF, DFDC and ASVspoof need forms or licences, and approval can take days. Use subsets: a few thousand samples per class is enough for fine-tuning.

Training rules

- Train on **Colab or Kaggle GPUs**, not in the CPU Docker container. Export weights to `models/`, which the worker reads.
- **Split by identity / source video**, never by random frame, or the model memorises faces and the scores look fake-good.
- **Augment for real life:** JPEG re-compression, resizing, blur, and WhatsApp-style re-encoding. This directly addresses the "forwarded many times" limitation.
- **Cross-dataset test:** e.g. train on FaceForensics++, test on Celeb-DF; train on ASVspoof, test on In-the-Wild. Report both numbers honestly.
- Metrics: **AUC** and accuracy for images and video, **EER** for audio (the ASVspoof standard), plus the rate of Inconclusive answers.
- Save every experiment's config and results in `ml/` and summarise them in MEMORY.md.

6. Backend and frontend details

API endpoints

Endpoint	Purpose
POST /api/analyses	Upload a file → returns analysis id; job is queued
GET /api/analyses/{id}	Status (queued / processing / done / failed) and full result
GET /api/analyses	History list with thumbnail, verdict, date
GET /api/analyses/{id}/report	Download the PDF report
GET /api/analyses/{id}/heatmap	Heatmap image
DELETE /api/analyses/{id}	Delete the file and its result (privacy promise)

Pages

- **Upload:** drag and drop, file-type hints, size limit message.
- **Progress:** step indicator (preprocessing → image → audio → combining), polling every 1-2 s.

- **Result:** verdict card, confidence meter, top reasons, heatmap viewer, video/audio player with a clickable fake-segment timeline, per-detector evidence accordion, Download PDF button.
- **History:** table with thumbnails, filters and delete.

Engineering must-haves

- Load models **once per worker process**, not per request.
- Generated file names only (never trust the uploaded name); auto-delete uploads after N days.
- Timeouts and clear error states ("Video too long: maximum 2 minutes for the demo").
- Tests: unit tests for fusion and each detector on small sample files; one end-to-end test through docker compose.

7. Roadmap

Estimates assume one developer working most days. If the deadline is closer, follow the cut line in section 8.

Phase	Deliverable	Est.	Done when
0. Skeleton	Folders, Docker, health check, MEMORY.md	Done	docker compose up shows "API: ok"
1. Upload-to-result loop	Upload API, Celery job, metadata + ELA + FFT detectors, fusion, result page	3-4 days	Upload an edited JPEG and see a verdict, reasons and the ELA heatmap
2. Image model	CLIP linear probe for AI images + face classifier with Grad-CAM	3-4 days	AI images and face swaps are detected on a held-out test set
3. Audio	Anti-spoofing model, splice check, audio timeline UI	3-4 days	Cloned voice clip shows red segments on the timeline
4. Video	Frame sampling, face tracking, per-second scores, temporal check, soundtrack analysis	4-5 days	Face-swap video shows the fake seconds on the timeline
5. Report + history	ReportLab PDF, history page, delete	2 days	PDF downloads with verdict, reasons and evidence images
6. Fine-tune + evaluate	Augmented fine-tuning, cross-dataset tests, calibrated fusion	4-5 days	Metrics table filled in MEMORY.md
7. Polish + demo	UI polish, sample files, slides, video recording	2-3 days	Rehearsed 3-minute demo

8. If time is short (cut line)

- **Must have:** Phases 1-3 plus a simple PDF. Images and audio cover the biggest real-world harms (scam calls, fake images).
- **Video lite:** per-frame face classifier + soundtrack analysis only. Skip blink, head pose and lip-sync.
- **Skip fine-tuning:** use pretrained weights directly and present cross-dataset results as "future work".

9. Hackathon demo plan

- Prepare 6 files: a real photo, an AI-generated image, a spliced image, a real voice note, a cloned voice clip, and a face-swap video.
- Show the **reason** each time (heatmap, timeline, sentence), not just the label. This is what sets ForenSight apart.
- Show one **Inconclusive** case (e.g. a heavily compressed WhatsApp forward) and explain why honesty matters.
- End with the PDF report and the history page. Mention limitations and responsible use in one slide.

10. Risks and mitigations

Risk	Mitigation
Model fails on new generators	CLIP-based detector for generalisation; multiple independent checks; Inconclusive band; easy to add or replace detectors
Compressed social-media files	Compression augmentation; lower reliability for low-quality inputs
Dataset access delayed	Apply now; start with open datasets (CIFAKE, GenImage, WaveFake, In-the-Wild)
Slow video on CPU	Sample at most 64 frames, small models, cap video length, run in background worker
Overclaiming accuracy	Report cross-dataset metrics; state clearly that ForenSight assists, not replaces, human judgement
Privacy of uploads	No third-party uploads, auto-expiry, delete button, files stay inside Docker volumes

11. Immediate next steps

- Install Docker Desktop and confirm `docker compose up --build` shows "API: ok".
- Apply for FaceForensics++, Celeb-DF, DFDC and ASVspoof access.
- Start Phase 1: upload API, Celery job, metadata + ELA + FFT detectors, result page.
- Log the hackathon deadline in MEMORY.md so the roadmap can be adjusted.